Title: Divide-and-Conquer Algorithms | Implementation, Experiments, and UI

Course: Design and Analysis of Algorithms

Submitted by:

- Muhammad Muzammil Siddiqui (23k-2001)
- Azaan Hussain (23k-0568)
- Rayyan Ali (23k-0532)

Instructor: Dr. Fahad Sherwani

Abstract

This project explores the implementation, evaluation, and visualization of classical divide-and-conquer algorithms. Two major algorithms were developed: the Closest Pair of Points algorithm in 2D and the Karatsuba fast integer multiplication algorithm. To support empirical analysis, random datasets of varying sizes were generated programmatically. A React-based user interface was designed to load input files, run algorithms, and visually present outputs using plots and algorithm logs. The report outlines the system architecture, dataset structure, execution setup, correctness verification through brute-force methods, and complexity analysis. Together, these components form a complete environment for both theoretical and experimental study of divide-and-conquer strategies.

Introduction

Divide-and-conquer is a core algorithmic technique in which a problem is divided into smaller subproblems, solved recursively, and then merged to form the final solution. This paradigm enables efficient algorithms for many computational tasks that are otherwise expensive when solved using brute-force methods.

This project focuses on two representative divide-and-conquer problems from the course textbook:

1. Closest Pair of Points in the 2D plane
2. Karatsuba Fast Integer Multiplication

Both algorithms provide substantial performance improvements over their naive counterparts. Alongside the theoretical implementations, we created dataset generators, correctness verifiers, and a complete visualization interface. The goal was not only to implement the algorithms but also to analyze their behavior on real input data and provide an interactive tool for experimentation.

Proposed System

Architecture Diagram

System Overview

The system consists of four major subsystems:

1. Input Layer
    - Accepts .txt files containing either point coordinates or large integers.
    - Closest-pair files contain one point per line: "x y".
    - Karatsuba files contain two integers, each 100–500 digits long.
2. Parser and Validator
    - Ensures correct input formatting.
    - Converts raw text into structured arrays of points or BigInt-compatible strings.
    - Provides descriptive error messages for malformed input.

3. Algorithm Engine
   Implemented in both Python (for experiments) and JavaScript (for UI execution), it includes:
   - Closest Pair (Divide & Conquer)
     Recursively splits the point set, solves each half, and merges by checking a central strip.
   - Karatsuba Multiplication
     Recursively splits integer strings into high/low parts and computes the product using
     ($T(n) = 3T(n/2) + O(n)$).
   - Brute-Force Verifiers
     Used for correctness checking on small datasets (≤300 points for closest pair).
4. User Interface & Visualization
   - Built using React and Plotly.
   - Allows file upload, algorithm selection, real-time logs, and plot-based visualization.
   - For closest pair, all points are plotted and the closest pair is highlighted.
   - For Karatsuba, the final product and digit count are displayed.

Experimental Setup

Dataset Generation

Two sets of datasets were generated using Python:

1. Closest-Pair Point Sets

Ten files were created using fixed seeds to ensure reproducibility:

File	Size
points_1.txt	120 points
points_2.txt	150
points_3.txt	200
points_4.txt	300
points_5.txt	400
points_6.txt	600
points_7.txt	800

File	Size
points_8.txt	1000
points_9.txt	1500
points_10.txt	2000

Each point is sampled uniformly from ([-1000, 1000]).

2. Karatsuba Integer Files

Files int_1.txt to int_10.txt, each containing:

- Integer 1: length 100–500 digits
- Integer 2: length 100–500 digits

Generated via a custom generate_integer() function.

Execution Commands

Closest Pair Batch Run

python3 run_closest_pair_all.py

Single File Execution

python3 -c "from closest_pair import closest_pair; print(closest_pair([...]))"

Karatsuba Batch Run

python3 run_karatsuba_all.py

Results and Discussion

Theoretical Complexity

- Closest Pair (D&C):
  (O(n \log n)). Splitting and recursive calls dominate; the merge step checks only a small strip.
- Brute-Force Closest Pair:
  (O(n^2)). All point pairs are checked.
- Karatsuba Multiplication:
  (T(n) = 3T(n/2) + O(n) = O(n^{1.585})).
  Faster than the (O(n^2)) grade-school algorithm.

Empirical Results

(Insert measured values once you run your scripts.)

Closest Pair Results Example Template

File	Points	Algorithm	Distance	Time (s)	Brute-Force Check
points_1.txt	120	D&C	0.123456	0.00XXX	MATCHED
points_4.txt	300	D&C	0.023456	0.00YYY	MATCHED
points_10.txt	2000	D&C	0.002345	0.0ZZZ	N/A

Karatsuba Results Example Template

File	len(x)	len(y)	Output Digits	Time (s)
int_1.txt	456	389	845	0.AAA

Interpretation

As expected, the divide-and-conquer closest-pair algorithm scales efficiently as input size grows, while brute-force becomes computationally prohibitive for large datasets. Karatsuba multiplication consistently reduced computation time for longer integers and demonstrated its subquadratic growth behavior. Together, the empirical results align well with the theoretical complexity analysis.

Conclusion

This project successfully implemented and analyzed divide-and-conquer algorithms through a combination of dataset generation, correctness verification, performance evaluation, and user-friendly visualization. The integration of Python experiments with a React-based UI created a comprehensive platform for algorithmic exploration. The system is fully reproducible due to fixed seeds and modular code, and it offers clear insights into the effectiveness of divide-and-conquer techniques for classic computational problems.

References

1. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd Edition.
2. J. Kleinberg and É. Tardos, *Algorithm Design*.
3. Python Documentation – math.hypot, random.
4. Plotly.js Documentation